VITYARTHI | BUILD YOUR OWN PROJECT

MoodFit

Mood-Based Fashion Recommendation System

Project Report

SUBMITTED BY
Ridhima Mohan Sinha
REGISTRATION NO.
26BEC10061
COURSE
B.Tech in Electronic and Telecommunications Engineering
INSTITUTION
VIT Bhopal

LANGUAGE
Python 3 | Console application

Contents

1	Cover Page	1
2	Introduction	2
3	Problem Statement	2
4	Functional Requirements	3
5	Non-Functional Requirements	3
6	System Architecture	4
7	Design Diagrams	5-6
8	Design Decisions & Rationale	7
9	Implementation Details	7
10	Screenshots / Results	8
11	Testing Approach	8
12	Challenges Faced	9
13	Learnings & Key Takeaways	9
14	Future Enhancements	9
15	References	10

2 Introduction

Choosing what to wear is rarely a purely practical decision. Mood, weather, and the occasion all influence the outfit a person finally settles on, yet most fashion tools recommend clothes only by trend or by category. **MoodFit** is a Python-based, console-driven fashion recommendation system that treats **how the user feels** as a first-class input. By combining mood with weather, occasion, and preferred style, it produces a suggestion that is personal, comfortable, and appropriate.

The application is intentionally built as a set of five small, independent modules coordinated by a single entry point (`main.py`). Each module owns exactly one step of the user journey - collecting user details, capturing mood, capturing weather, capturing occasion and style, and generating the final recommendation. This separation makes the code easy to read, test, and extend.

5	**4**	**1**	**CLI**
Independent modules	Recommendation inputs (mood, weather, occasion, style)	Clear entry point (main.py)	Lightweight, no setup

3 Problem Statement

People frequently struggle to decide what to wear because the decision depends on several factors at once - emotional state, current weather, the nature of the event, and personal style. Existing tools usually consider only one of these factors, leading to outfits that look right but feel wrong, or suit the event but not the weather.

> **Problem:** There is no simple, accessible tool that combines a person's **mood**, the **weather**, the **occasion**, and their preferred **style** into one coherent outfit recommendation.

Objectives

- Collect essential user information and preferences through a clear, guided console interface.
- Model mood, weather, occasion, and style as distinct inputs to the recommendation.
- Generate a relevant outfit suggestion from the combination of these inputs.
- Deliver a modular, well-documented, maintainable codebase that can be extended easily.

4 Functional Requirements

MoodFit provides five functional modules (the guidelines require a minimum of three). Each has a defined input and a defined output, and together they form a single, logical workflow.

ID	Module	Function	Input	Output
FR1	User Information user_information.py	Captures the user's name and age.	Name, age (typed)	(name, age)
FR2	Mood Selection mood_module.py	Lets the user choose their current mood.	Mood option	mood
FR3	Weather Selection weather_module.py	Captures the prevailing weather condition.	Weather option	weather
FR4	Occasion & Style clothing_module.py	Captures the event type and the preferred clothing style.	Occasion, style	(occasion, style)
FR5	Recommendation recommendation_module.py	Combines all inputs and produces a personalised outfit suggestion.	name, age, mood, weather, occasion, style	Outfit recommendation on screen

Workflow. The user launches the program, is greeted by a welcome banner, and is guided step by step through FR1 to FR5. The final recommendation is displayed as the last step.

5 Non-Functional Requirements

Six non-functional requirements shape the quality of the solution (the guidelines require at least four).

Requirement	Description	How MoodFit addresses it
Usability	Interaction must be obvious for a first-time user.	Guided prompts, one question at a time, plain-language options, clear banner.
Maintainability	Code should be easy to understand and change.	One responsibility per module; small functions; comments; a thin main.py.
Reliability	Program should not crash on wrong input.	Input validation with re-prompting so the flow always reaches a recommendation.
Performance	Response should feel instant.	Lightweight in-memory processing; no network or database calls.
Scalability	New moods, weather types, or styles should be easy to add.	Options are defined per module, so extending a category touches one file.
Resource efficiency	Minimal CPU, memory and setup.	Pure Python standard library; no external dependencies to install.

6 System Architecture

MoodFit follows a **modular, controller-based architecture**. `main.py` acts as the controller: it owns the order of execution and passes data between modules, while each module is responsible for a single concern. Data is held in memory for the duration of one session and is never written to disk.

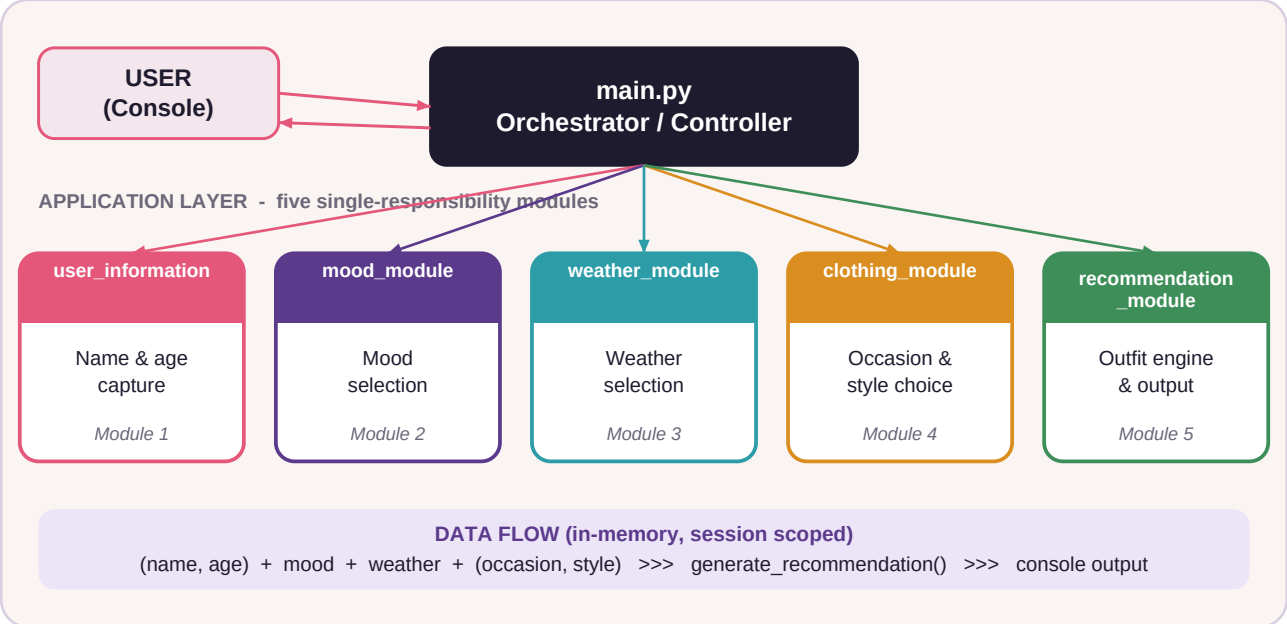


Figure 1: System architecture of MoodFit

Architectural Layers

Layer	Components	Responsibility
Interface layer	Console (input / print)	Shows prompts and results; receives typed responses from the user.
Control layer	main.py	Imports the five modules and calls them in sequence; hands collected values to the recommendation step.
Functional layer	Five modules	Each module collects or produces one piece of information and returns it to the controller.
Data	In-memory variables	name, age, mood, weather, occasion, style are held as local variables for the session.

Why this works: because modules communicate only through the values returned to `main.py`, any module can be rewritten or replaced without affecting the others.

7 Design Diagrams

7.1 Use Case Diagram

The single actor, the **User**, interacts with the five use cases of the system. Every data-entry use case includes an input-validation step.

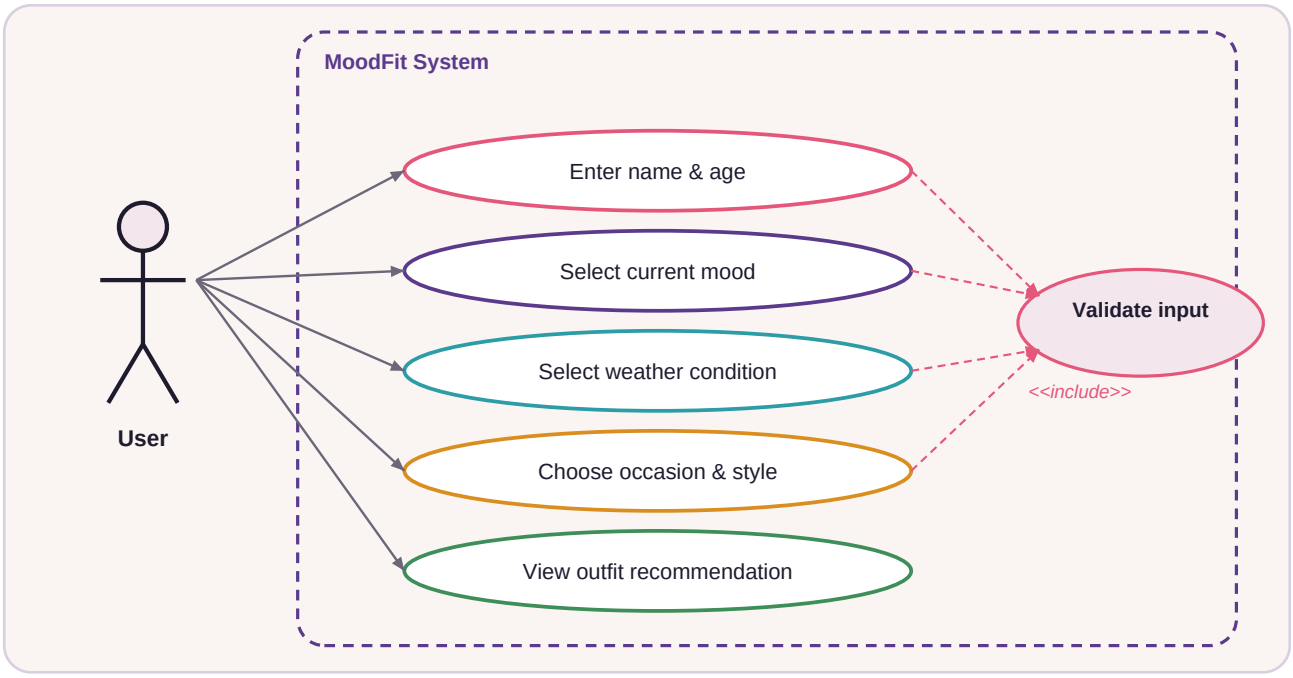


Figure 2: Use case diagram

7.2 Workflow Diagram

The process runs strictly in one direction, with re-prompting at each input step until valid data is supplied.

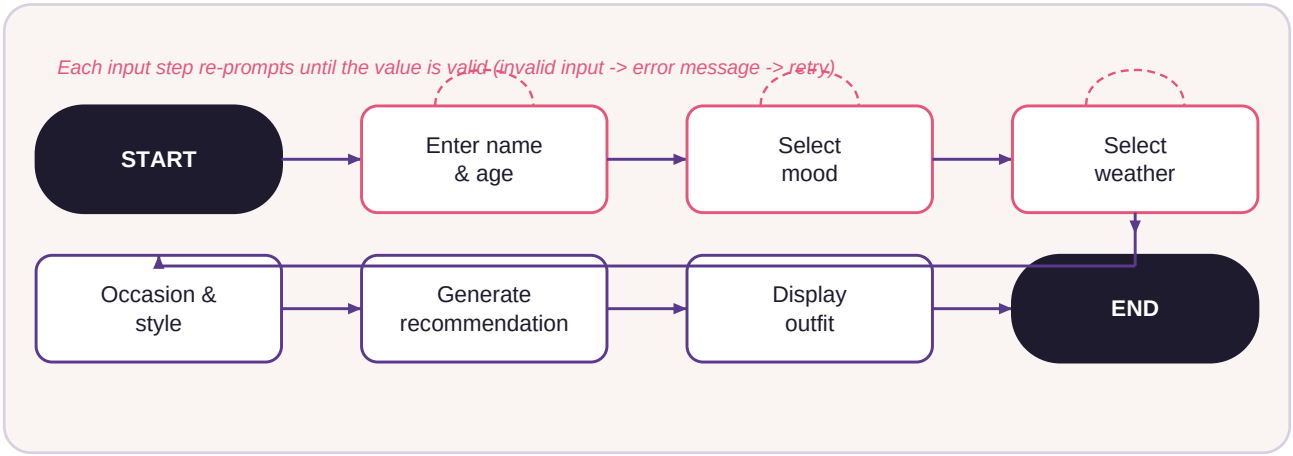


Figure 3: Process / workflow diagram

7.3 Sequence Diagram

The sequence below shows the order of calls between the user, the controller, and each module during one complete run.

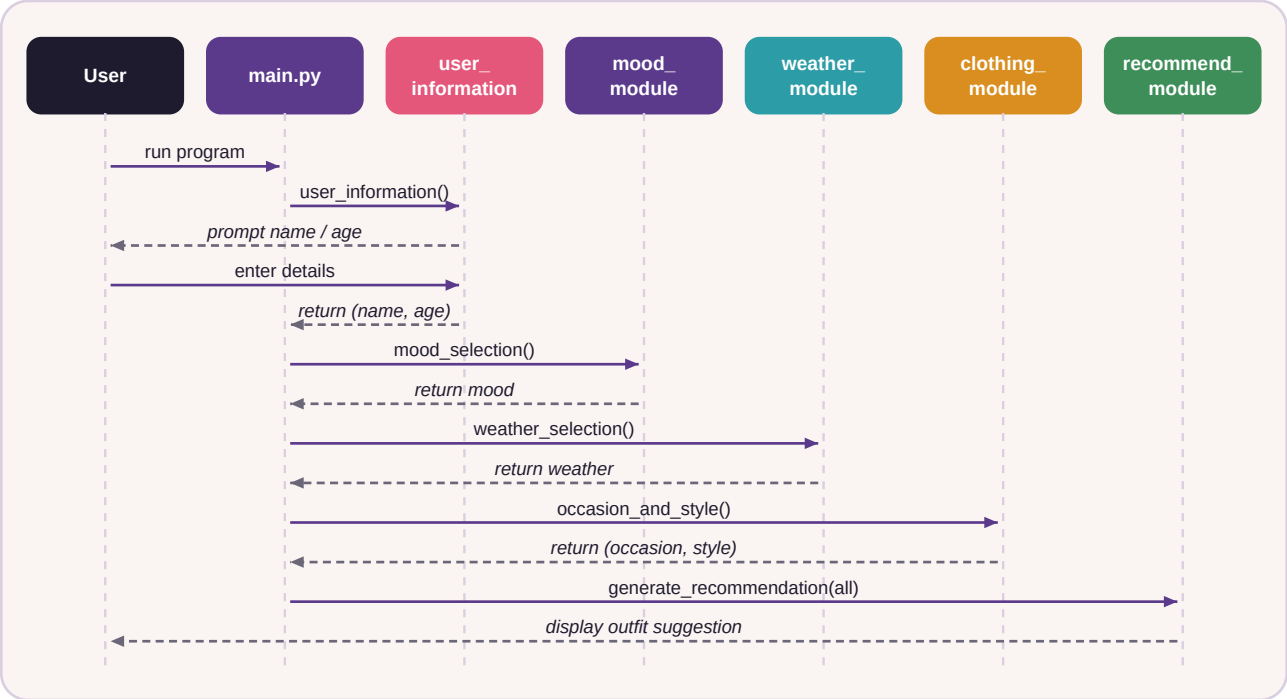


Figure 4: Sequence diagram of a single session

7.4 Component / Class Diagram

MoodFit uses a functional (module-based) design, so the class diagram is presented as a component diagram: each module exposes one public function.

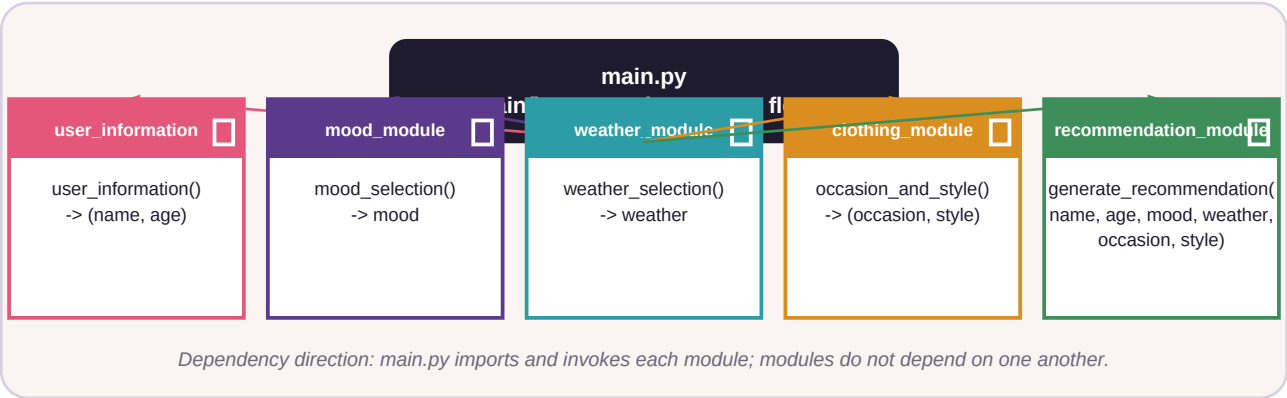


Figure 5: Component diagram

7.5 ER Diagram

Not applicable. MoodFit does not use a database or persistent storage; all data lives in memory for the duration of a session. An ER diagram would become relevant if user profiles or outfit history were stored in a future version (see Section 14).

8 Design Decisions & Rationale

Decision	Rationale
Five single-purpose modules	Separates concerns, makes each step easy to test in isolation, and satisfies the multi-module structure expected of the project.
Thin controller (main.py)	Keeps program flow visible in one place; the entry point reads almost like a description of the user journey.
Data returned, not shared	Modules return values instead of using global state, so they stay decoupled and predictable.
Console interface	Focuses effort on the logic and design rather than UI frameworks; runs anywhere Python runs.
Mood as a primary input	The key point of difference of MoodFit: emotional context shapes the outfit alongside weather and occasion.
No external dependencies	Simplifies installation and evaluation - the project runs with a plain Python install.

9 Implementation Details

The project is implemented in Python 3 with a flat, readable structure. The program starts in `main.py`, which prints the welcome banner and then invokes each module in order:

```
name, age = user_information()
mood = mood_selection()
weather = weather_selection()
occasion, style = occasion_and_style()
generate_recommendation(name, age, mood, weather, occasion, style)
```

Listing 1: The main() workflow in main.py

Module	Public function	Role in the program
user_information.py	user_information()	Prompts for and returns the user's name and age.
mood_module.py	mood_selection()	Presents the mood choices and returns the selected mood.
weather_module.py	weather_selection()	Presents the weather choices and returns the selected condition.
clothing_module.py	occasion_and_style()	Collects the occasion and preferred style and returns both.
recommendation_module.py	generate_recommendation(...)	Receives all six values, maps them to an outfit suggestion, and prints the result.

10 Screenshots / Results

The screens below show the program flow. Replace the highlighted areas with screenshots from your own run before submission.

```
$ python main.py
WELCOME TO MOODFIT
Mood-Based Fashion Recommendation System
[ Screenshot 1 - user details, mood and weather prompts ]
```

Figure 6: Program start and input prompts

```
[ Screenshot 2 - occasion, style prompts ]
[ Screenshot 3 - final outfit recommendation output ]
```

Figure 7: Recommendation output

11 Testing Approach

Testing is done by running the program end to end with representative and edge-case inputs, verifying that each module returns the expected value and that the final recommendation reflects the inputs. The planned test cases are:

ID	Scenario	Input	Expected outcome
T1	Normal run	Valid name, age, mood, weather, occasion, style	Recommendation is displayed
T2	Different moods	Same inputs, change mood only	Recommendation changes with mood
T3	Different weather	Same inputs, change weather only	Recommendation adapts to weather
T4	Invalid option	Out-of-range menu choice	Error message and re-prompt
T5	Invalid age	Non-numeric or negative age	Rejected; user asked again
T6	Empty input	Blank name or choice	Rejected; user asked again

12 Challenges Faced

- **Designing clean module boundaries.** Deciding exactly what each module should own, and what it should return, so that modules stayed independent of each other.
- **Handling invalid user input.** Making sure typos, out-of-range choices and empty entries never crash the program or corrupt the recommendation.
- **Combining several inputs sensibly.** Ensuring the mood, weather, occasion and style together lead to a suggestion that feels coherent, not contradictory.
- **Keeping the flow simple.** Balancing enough questions for a good recommendation against a short, pleasant user experience.

13 Learnings & Key Takeaways

- Modular design makes a program far easier to reason about, debug and extend than a single long script.
- Clear function inputs and outputs act as contracts between modules.
- Validating input early prevents a whole class of errors later in the program.
- Documenting requirements and diagrams before and during implementation keeps the design consistent.
- Version control (Git) gives a safe history of changes and makes iteration less risky.

14 Future Enhancements

Live weather Fetch real-time weather from a weather API instead of manual selection.	**Web or mobile UI** Replace the console with a friendly graphical interface.
User profiles & history Store preferences and past outfits in a database (adds an ER design).	**Machine learning** Learn from user feedback to improve the recommendation over time.
Wardrobe integration Recommend from clothes the user actually owns.	**Shopping links & images** Show outfit visuals and suggest where to buy.

Project Repository Structure

The GitHub repository is organised as follows, with the required README.md and statement.md at the root:

```
MoodFit/
|-- main.py                    (entry point / controller)
|-- user_information.py        (module 1)
|-- mood_module.py             (module 2)
|-- weather_module.py          (module 3)
|-- clothing_module.py         (module 4)
|-- recommendation_module.py   (module 5)
|-- README.md                  (overview, install, run, testing)
|-- statement.md               (problem statement, scope, users)
```

How to run: install Python 3, open a terminal in the project folder, and run `python main.py`.

Conclusion

MoodFit demonstrates how a modular design, clear data flow, and a user-centred idea can be combined into a compact yet complete application. By placing mood alongside weather, occasion and style, it offers a more human approach to outfit selection. The project meets the guideline requirements of at least three functional modules, a clear input/output structure and logical workflow, and lays a solid foundation for enhancements such as live weather, user profiles, and learning-based recommendations.

15 References

- Python Software Foundation. *Python 3 Documentation*. https://docs.python.org/3/
- Sommerville, I. *Software Engineering*. Pearson - for requirements engineering and architectural design concepts.
- Object Management Group. *Unified Modeling Language (UML) Specification*. https://www.omg.org/spec/UML/
- Pro Git Book. *Git Basics and Branching*. https://git-scm.com/book
- VITyarthi. *Build Your Own Project - General Instructions & Submission Guidelines*.